

DataFrames, Datasets & Spark SQL

Spark SQL and its DataFrames and Datasets interfaces are the future of Spark performance, with more efficient storage options, advanced optimizer, and direct operations on serialized data. These components are super important for getting the best of Spark performance - see [Figure 3-1](#).

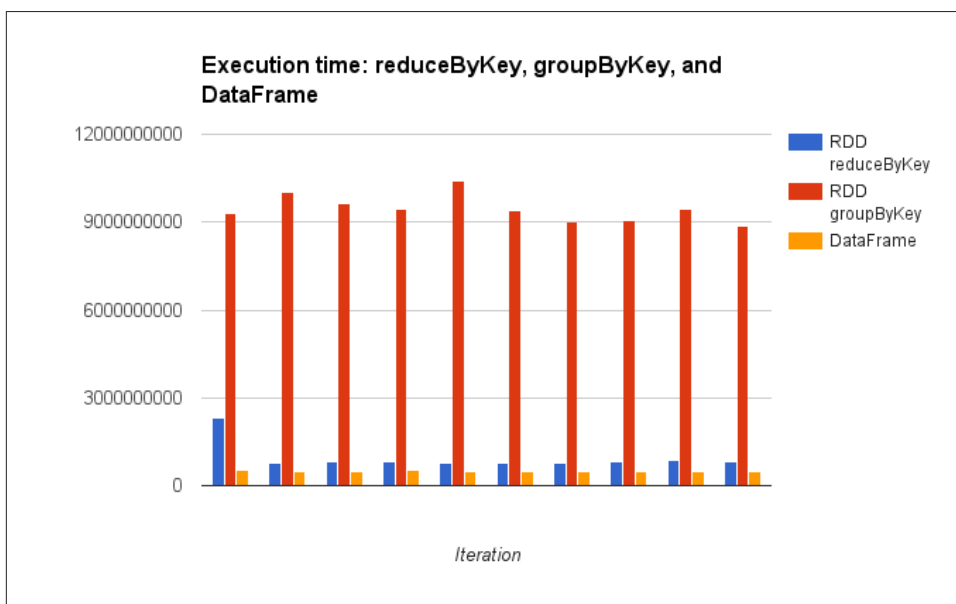


Figure 3-1. Spark SQL Performance Relative Simple RDDs From SimplePerfTest.scala Aggregating avg Fuzziness

These are relatively new components; Datasets was introduced in Spark 1.6, DataFrames in Spark 1.3, and the SQL engine in Spark 1.0. This chapter is focused on

helping you learn how to best use Spark SQL's tools. For tuning params a good follow up is ???.



Spark's DataFrames have very different functionality compared to traditional DataFrames like Panda's and R. While these all deal with structured data, it is important not to depend on your existing intuition surrounding DataFrames.

Like RDDs, DataFrames and Datasets represent distributed collections, with additional schema information not found in RDDs. This additional schema information is used to provide a more efficient storage layer and in the optimizer. Beyond schema information, the operations performed on DataFrames are such that the optimizer can inspect the logical meaning rather than arbitrary functions. Datasets are an extension of DataFrames bringing strong types, like with RDDs, for Scala/Java and more RDD-like functionality within the Spark SQL optimizer. Compared to working with RDDs, DataFrames allow Spark's optimizer to better understand our code and our data, which allows for a new class of optimizations we explore in “[Query Optimizer](#)” on page 75.



While Spark SQL, DataFrames, and Datasets provide many excellent enhancements, they still have some rough edges compared to traditional processing with “regular” RDDs. The Dataset API, being brand new at the time of this writing, is likely to experience some changes in future versions.

Getting Started with the HiveContext (or SQLContext)

Much as the SparkContext is the entry point for all Spark applications, and the StreamingContext is for all streaming applications, the HiveContext and SQLContext serve as the entry points for Spark SQL. The names of these entry points can be a bit confusing, and it is important to note the HiveContext **does not require** a Hive installation. The primary reason to use the SQLContext is if you have conflicts with the Hive dependencies that cannot be resolved. The HiveContext has a more complete SQL parser as well as additional user defined functions (UDFs) ⁵ and should be used whenever possible.

Like with all of the Spark components, you need to import a few extra components as shown in [Example 3-1](#). If you are using the HiveContext you should import those

⁵ UDFs allow us to extend SQL to have additional powers, such as computing the geo-spatial distance between points

components as well. Once you have imported the basics, you can create the entry point as in [Example 3-2](#).

Example 3-1. Spark SQL imports

```
import org.apache.spark.sql.{DataFrame, SQLContext, Row}
import org.apache.spark.sql.catalyst.expressions.aggregate._
import org.apache.spark.sql.expressions._
import org.apache.spark.sql.functions._
// Additional imports for using HiveContext
import org.apache.spark.sql.hive._
import org.apache.spark.sql.hive.thriftserver._
```

Example 3-2. Creating the HiveContext

```
val hiveContext = new HiveContext(sc)
// Import the implicits, unlike in core Spark the implicits are defined on the context
import hiveContext.implicits._
```

To use the HiveContext you will need to add both Spark's SQL and Hive components to your dependencies. If you are using the [sbt-spark-package](#) plugin you can do this by just adding SQL and Hive to your list of sparkComponents.

Example 3-3. Add Spark SQL & Hive component to sbt-spark-package build

```
sparkComponents += Seq("sql", "hive")
```

For maven compatible build systems, the coordinates for Spark's SQL and Hive components in 1.6.0 are `org.apache.spark:spark-sql_2.10:1.6.0` and `org.apache.spark:spark-hive_2.10:1.6.0`.

Example 3-4. Add Spark SQL & Hive component to “regular” sbt build

```
libraryDependencies += Seq(
  "org.apache.spark" %% "spark-sql" % "1.6.0",
  "org.apache.spark" %% "spark-hive" % "1.6.0")
```

Example 3-5. Add Spark SQL & Hive component to maven pom file

```
<dependency> <!-- Spark dependency -->
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.10</artifactId>
  <version>1.6.0</version>
</dependency>
<dependency> <!-- Spark dependency -->
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-hive_2.10</artifactId>
```

```
<version>1.6.0</version>
</dependency>
```

While it's not required, if you do have an existing Hive Metastore to which you wish to connect your HiveContext, you can copy your `hive-site.xml` to Spark's `conf/` directory.



As of Spark 1.6.0, the default Hive Metastore version is 1.2.1. For other versions of the Hive Metastore you will need to set the `spark.sql.hive.metastore.version` property to the desired versions as well as set `spark.sql.hive.metastore.jars` to either “maven” (to have Spark retrieve the jars) or the system path where the Hive JARs are present.

If you can't include the Hive dependencies with our application, you can leave out Spark's Hive component and instead create a `SQLContext`. This provides much of the same functionality, but uses a less capable SQL parser and lacks certain Hive based user defined functions (UDFs) and user defined aggregate functions (UDAFs).

Example 3-6. Creating the SQLContext

```
val sqlContext = new SQLContext(sc)
// Import the implicits, unlike in core Spark the implicits are defined on the context
import sqlContext.implicits._
```



If you are using the Spark Shell you will automatically get a `SQLContext`. This `SQLContext` will be a `HiveContext` if Spark was built with Hive support (e.g. `-PHive`).

As with the core Spark Context and Streaming Context, the `Hive/SQLContext` is used to load your data. JSON format data is widely popular, and some sample data about pandas, as well as some complex data for the Goldilocks example, is included in this book's sample resources directory. JSON is especially interesting since it lacks schema information, and Spark needs to do some work to infer the schema from our data. We will cover the full loading and saving API for JSON in “JSON” on page 59, but to get started let's load a sample we can use to explore the schema.

Example 3-7. Load JSON sample

```
val df = sqlCtx.read.json(path)
```

Basics of Schemas

The schema information, and the optimizations it enables, is one of the core differences between Spark SQL and core Spark. Inspecting the schema is especially useful for DataFrames since you don't have the templated type you do with RDDs or DataSets. Schemas are normally handled automatically by Spark SQL, either inferred when loading the data or computed based on the parent DataFrames and the transformation being applied.

DataFrames expose the schema in both human-readable or programmatic formats. `printSchema()` will show us the schema of a DataFrame and is most commonly used when working in the shell to figure out with what you are working. This is especially useful for data formats, like in JSON, where the schema may not be immediately visible by looking at only a few records or reading a header. For programmatic usage, you can get the schema by simply calling `schema`, which is often used in ML pipeline transformers. Since you are likely familiar with case classes and JSON, let's examine how the equivalent Spark SQL schema would be represented.

Example 3-8. JSON data which would result in an equivalent schema

```
{"name": "mission", "pandas": [{"id": 1, "zip": "94110", "pt": "giant", "happy": true,
                                "attributes": [0.4, 0.5]}]}
```

Example 3-9. Equivalent case class to the next two examples

```
case class RawPanda(id: Long, zip: String, pt: String, happy: Boolean, attributes: Array[Double])
case class PandaPlace(name: String, pandas: Array[RawPanda])
```

Example 3-10. StructField case class

```
case class StructField(
  name: String,
  dataType: DataType,
  nullable: Boolean = true,
  metadata: Metadata = Metadata.empty)
....
```

Example 3-11. Sample schema information for nested structure (schema) - manually formatted

```
org.apache.spark.sql.types.StructType = StructType(
  StructField(name,StringType,true),
  StructField(pandas,
    ArrayType(
      StructType(StructField(id,LongType,false),
        StructField(zip,StringType,true),
        StructField(pt,StringType,true),
```

```
StructField(happy, BooleanType, false),
StructField(attributes, ArrayType(DoubleType, false), true)), true), true))
```

Example 3-12. Sample Schema Information for Nested Structure (printSchema)

```
root
|-- name: string (nullable = true)
|-- pandas: array (nullable = true)
|   |-- element: struct (containsNull = true)
|   |   |-- id: long (nullable = false)
|   |   |-- zip: string (nullable = true)
|   |   |-- pt: string (nullable = true)
|   |   |-- happy: boolean (nullable = false)
|   |   |-- attributes: array (nullable = true)
|   |   |   |-- element: double (containsNull = false)
```

From here we can dive into what this schema information means and look at how to construct more complex schemas. The first part is a `StructType` which contains a list of fields. It's important to note you can nest `StructTypes`, like how a case class can contain additional case classes. The fields in the `StructType` are defined with `StructField` which specifies the name, type (see [Table 3-1](#) and [Table 3-2](#) for a listing of types), and a boolean indicating if the field may be null/missing.

Table 3-1. Basic Spark SQL types

Scala Type	SQL Type	Details
Byte	ByteType	1 byte signed integers (-128,127)
Short	ShortType	2 byte signed integers (-32768,32767)
Int	IntegerType	4 byte signed integers (-2147483648,2147483647)
Long	LongType	8 byte signed integers (-9223372036854775808,9223372036854775807)
java.math.BigDecimal	DecimalType	Arbitrary precision signed decimals
Float	FloatType	4 byte floating point number
Double	DoubleType	8 byte floating point number
Array[Byte]	BinaryType	Array of bytes
Boolean	BooleanType	true/false
java.sql.Date	DateType	Date without time information
java.sql.Timestamp	TimestampType	Date with time information (second precision)

Scala Type	SQL Type	Details
String	StringType	Character string values (stored as UTF8)

Table 3-2. Complex Spark SQL types

Scala Type	SQL Type	Details	Example
Array[T]	ArrayType(elementType, containsNull)	Array of single type of element, containsNull true if any null elements.	Array[Int] => ArrayType(IntegerType, true)
Map[K, V]	MapType(elementType, valueType, valueContainsNull)	Key/value map, valueContainsNull if any values are null.	Map[String, Int] => MapType(StringType, IntegerType, true)
case class	StructType(List[StructFields])	Named fields of possible heterogeneous types, similar to a case class or JavaBean.	case class Panda(name: String, age: Int) => StructType(List(StructField("name", StringType, true), StructField("age", IntegerType, true)))



As you saw in [Example 3-11](#), you can nest StructFields and all of the complex Spark SQL types.

Now that you've got an idea of how to understand and, if needed, construct schemas for, your data, you are ready to start exploring the DataFrame interfaces.



Spark SQL schemas are eagerly evaluated, unlike the data underneath. If you find yourself in the shell and uncertain of what a transformation will do - try it and print the schema. See [Example 3-12](#).

DataFrame API

Spark SQL's DataFrame API allows us to work with DataFrames without having to register temporary tables or generate SQL expressions. The DataFrame API has both transformations and actions. The transformations on DataFrames are more relational in nature, with the Dataset API (covered next) offering a more functional-style API.

Transformations

Transformations on DataFrames are similar in concept to RDD transformations, but with a more relational flavor. Instead of specifying arbitrary functions, which the optimizer is unable to introspect, you use a restricted expression syntax so the optimizer can have more information. As with RDDs, we can broadly break down transformations into single, multi, key/value, and grouped/windowed transformations.



Spark SQL transformations are only partially lazy; the schema is eagerly evaluated.

Simple DataFrame Transformations & SQL Expressions

Simple DataFrame transformations allow us to do most of the standard things one can do when working a row at a time.⁵ You can still do many of the same operations defined on RDDs, except using Spark SQL expressions instead of arbitrary functions. To illustrate this we will start by examining the different kinds of filter operations available on DataFrames.

DataFrame functions, like `filter` accept Spark SQL expressions instead of lambdas. These expressions allow the optimizer to understand what the condition represents, and with `filter`, it can often be used to skip reading unnecessary records.

To get started, let's look at a SQL expression to filter our data for unhappy pandas using our existing schema. The first step is looking up the column that contains this information. In our case it is "happy", and for our DataFrame (called `df`) we access the column through the `apply` function (e.g. `df("happy")`). The `filter` expression requires the expression to return a boolean value, and if you wanted to select happy pandas, the entire expression could be retrieving the column value. However, since we want to find the unhappy pandas though, we can check to see that happy isn't true using the `!==` operator as shown in [Example 3-13](#).

Example 3-13. Simple filter for unhappy pandas

```
pandaInfo.filter(pandaInfo("happy") !== true)
```

⁵ A row at a time allows for narrow transformations with no shuffle.



To lookup on the column, we can either provide the column name on the specific DataFrame or use the implicit \$ operator for column lookup. This is especially useful when the DataFrame is anonymous. The ! binary negation function can be used together with \$ to simplify our expression from [Example 3-13](#) down to `df.filter(!$("happy"))`.

This illustrates how to access a specific column from a DataFrame. For accessing other structures inside of DataFrames, like nested structs, keyed maps, and array elements, use the same apply syntax. So, if the first element in the attributes array represent squishiness, and you only want very squishy pandas, you can access that element by writing `df("attributes")(0) >= 0.5`.

Our expressions need not be limited to a single column. You can compare multiple columns in our “filter” expression. Complex filters like this are more difficult to push down to the storage layer, so you may not see the same speedup over RDDs that you see with simpler filters.

Example 3-14. More complex filter

```
pandaInfo.filter(pandaInfo("happy").and(
  pandaInfo("attributes")(0) > pandaInfo("attributes")(1)))
```



Spark SQL’s column operators are defined on the column class, so a filter containing the expression `0 >= df.col("friends")` will not compile since Scala will use the `>=` defined on 0. Instead you would write `df.col("friend") <= 0` or convert 0 to a column literal with `lit`⁵.

Spark SQL’s DataFrame API has a very large set of operators available. You can use all of the standard mathematical operators on floating points, along with the standard logical and bitwise operations (prefix with `bitwise` to distinguish from logical). Columns use `===` and `!==` for equality to avoid conflict with Scala internals. For columns of strings, `startsWith/endsWith`, `substr`, `like`, and `isNull` are all available. The full set of operations is listed in [org.apache.spark.sql.Column](#) and covered in [Table 3-3](#) and [Table 3-4](#).

⁵ A column literal is a column with a fixed value that doesn’t change between rows (i.e. constant).

Table 3-3. Spark SQL Scala operators

Scala Operator	Java Equivalent	Input Column Types	Output Type	Purpose	Sample	Result
!=	notEqual	Any	Boolean	Check if expressions not equal	"hi" != "bye"	true
%	mod	Numeric	Numeric	Modulo	10 % 5	0
&&	and	Boolean	Boolean	Boolean and	true && false	false
*	multiply	Numeric	Numeric	Multiply expressions	2 * 21	42
+	plus	Numeric	Numeric	Sum expression	2 + 2	4
-	minus	Numeric	Numeric	Subtraction	2 - 2	0
-	unary_-	Numeric	Numeric	Unary subtraction	-42	-42
/	division	Numeric	Double	Division	43/2	21.5
<	lt	Comparable	Boolean	Less than	"a" < "b"	true
<=	leq	Comparable	Boolean	Less than or equal to	"a" <= "a"	true
===	equals	Any	Any	Equality test (unsafe on null values)	"a" === "a"	true
<=>	eqNullSafe	Any	Any	Equality test (safe on null values)	"a" <=> "a"	true
>	gt	Comparable	Boolean	Greater than	"a" > "b"	false
>=	gt	Comparable	Boolean	Greater than or equal to	"a" >= "b"	false

Table 3-4. Spark SQL expression operators

Operator	Input Column Types	Output Type	Purpose	Sample	Result
apply	Complex types	Type of field accessed	Get value from complex type (e.g. structfield/map lookup or array index)	[1,2,3].apply(0)	1
bitwiseAND	Integral Type ^a	Same as input	Computes and bitwise	21.bitwiseAND(11)	1
bitwiseOR	Integral Type ^a	Same as input	Computes or bitwise	21.bitwiseOR(11)	31

Operator	Input Column Types	Output Type	Purpose	Sample	Result
bitwiseXOR	Integral Type ^a	Same as input	Computes bitwise exclusive or	21.bitwiseXOR(11)	30

^a Integral types include ByteType, IntegerType, LongType, and ShortType.



Not all Spark SQL expressions can be used in every API call. For example, Spark SQL joins do not support complex operations, and filter requires that the expression result in a boolean, and similar.

In addition to the operators directly specified on the column, an even larger set of functions on columns exists in [org.apache.spark.sql.functions](#), some of which we cover in tables [Table 3-5](#) and [Table 3-6](#). For illustration, this example shows the values for each column at a specific row, but keep in mind that, these functions are called on columns not values.

Table 3-5. Spark SQL standard functions

Function name	Purpose	Input Types	Example usage	Result
lit(value)	Convert a Scala symbol to a column literal ³	Column & Symbol	lit(1)	Column(1)
array	Create a new array column	Must all have the same Spark SQL type	array(lit(1),lit(2))	array(1,2)
isNaN	Check if not a number	Numeric	isNaN(lit(100.0))	false
not	Opposite value	Boolean	not(lit(true))	false

Table 3-6. Spark SQL Common Mathematical Expressions

Function name	Purpose	Input Types	Example usage	Result
abs	Absolute value	Numeric	abs(lit(-1))	1
sqrt	Square root	Numeric	sqrt(lit(4))	2
acos	Inverse cosine	Numeric	acos(lit(0.5))	1.04... ^a
asin	Inverse sine	Numeric	asin(lit(0.5))	0.523... ^a
atan	Inverse tangent	Numeric	atan(lit(0.5))	0.46... ^a